

Contrôle de types

Sommaire

Expressions de types

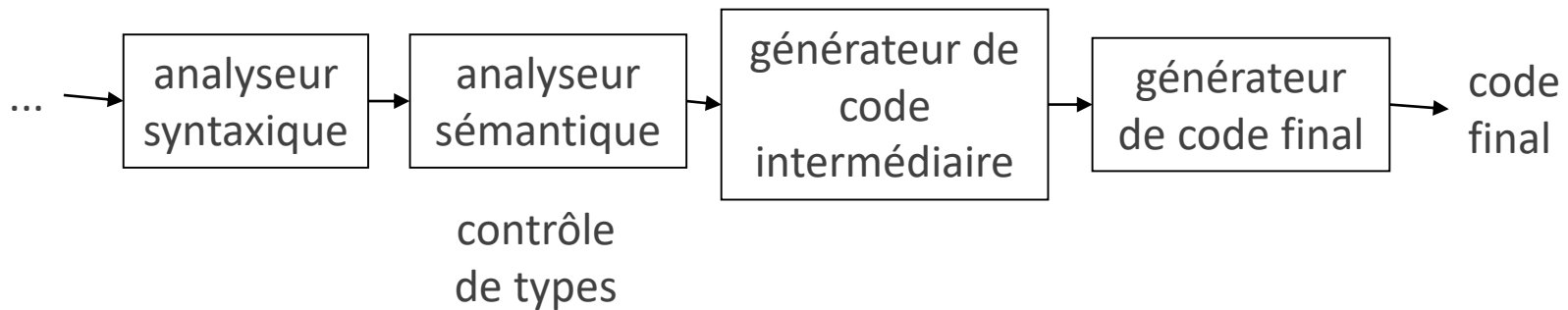
Un contrôleur de types

Déclarations de types en C

Conversions de types

Polymorphisme

Contrôle de types (*type checking*)



Les types sont une caractéristique des langages évolués

Pas de types en `nasm`

Représentation des structures de données

Langages à objets

Enrichissement des possibilités de typage

Contrôle de types

```
int check( int tab[], int size);  
(...)  
    if (!check( values[i], MAX))
```

Le contrôle de types détecte des erreurs

Exemple : comparer paramètre formel et
paramètre effectif

Quelles sont les informations fournies par le type d'une donnée ?

- à quoi elle sert
- taille en mémoire
- opérations applicables

Typage des expressions

```
typedef struct list{  
    STEntry content;  
    struct list *next;  
} list;  
(...)  
if (cell->next->next)
```

Le développeur déclare le type des variables
Et le type des expressions ?

Pour calculer le type d'une expression, l'arbre
abstrait de l'expression doit avoir un sous-
arbre pour chaque sous-expression

Typage des expressions

Typage statique

Le compilateur calcule le type des expressions

Réalisable avant l'exécution

Exemple : C

Avantages :

- sécurité une fois que le code est compilé
- compilation plus simple

Typage dynamique

L'exécutable calcule le type des expressions

Réalisable seulement pendant l'exécution

Exemples : Python, Java

Avantages : le même code est compatible avec plusieurs types

Pas de typage parfait

Exemple : division par zéro

```
typedef struct list{
    STEntry content;
    struct list *next;
} list;
(...)
if (cell->next->next)
```

Typage des expressions

Projet logiciel 2025-2026

Types simples `int` (4 octets) et `char` (1 octet)

Vérifier la validité des expressions

Tenir compte des conversions implicites

Exemple : avertissement si on passe un entier à
`putchar()`

`putchar('Z'+0);`

Représenter les types pendant la compilation

Comment indiquer le type d'une variable dans la table des symboles ?

```
int size;  
struct satellite gps;  
int tab[MAX];  
char *cursor;  
char *cursors[MAX];
```

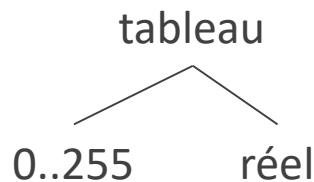
Types simples

Types complexes

Expressions de type

Expressions représentant les types des objets

Expressions de types



Types de base

booléen, caractère, entier, réel... et type vide

Constructeurs de types

tableau, pointeur...

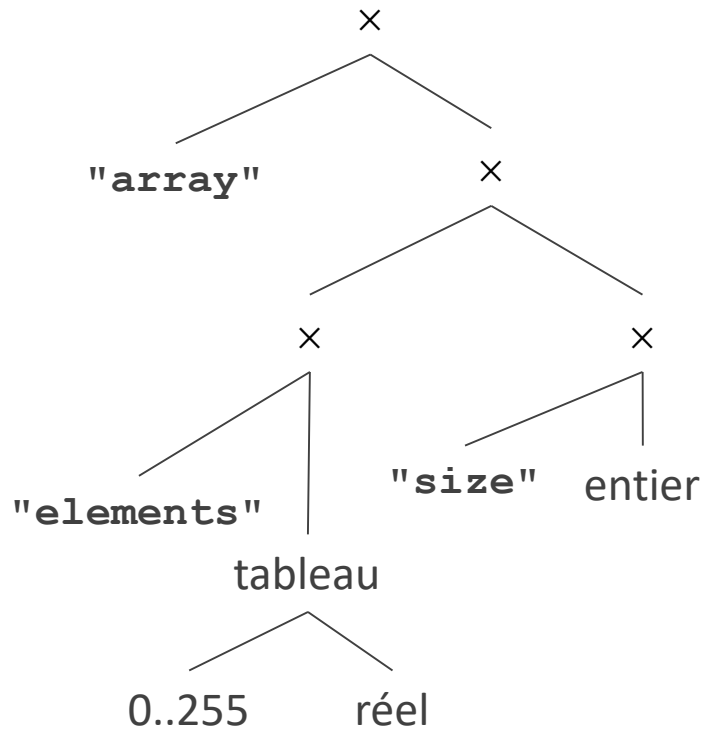
Les expressions de type peuvent être des arbres

Noms de types

nécessaires pour les types à définition récursive
(exemple : liste chaînée)

Variables de types

nécessaires pour les types génériques (fonctions
sur les types)



Constructeurs de types

Tableau

$\text{tableau}(I, T)$: tableaux d'éléments de type T
indiqués par des indices de type I

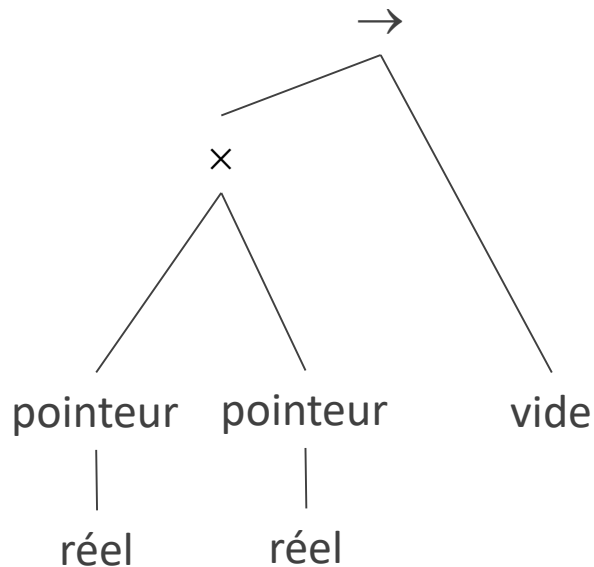
Produit

$T_1 \times T_2$: couples d'un élément de type T_1 et d'un
de type T_2

Structure

De même plus des noms pour les champs
et pour la structure

Constructeurs de types



Pointeur

pointeur(T) : pointeurs sur un objet de type T

Fonction

Les fonctions ont des types aussi

En C, le prototype est la déclaration du type de la fonction

$D \rightarrow R$: fonctions de D (type du paramètre) dans R (type de la valeur de retour)

Sommaire

Expressions de types

Un contrôleur de types

Déclarations de types en C

Conversions de types

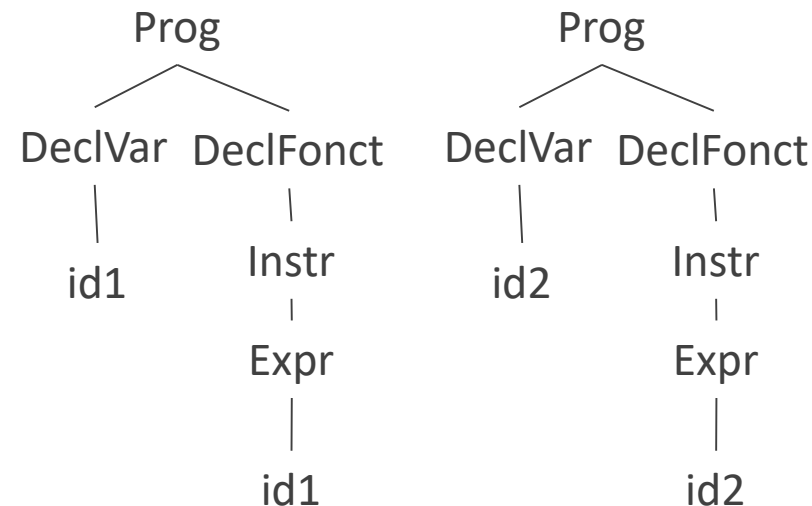
Polymorphisme

Les erreurs de typage sont des erreurs sémantiques

Erreurs lexicales : détectées par l'analyseur lexical

Erreurs syntaxiques : détectées par la grammaire

Erreurs sémantiques : les autres



Peut-on inclure le contrôle de types dans la syntaxe et dans la grammaire ?

Une grammaire qui interdirait `18%5.2`
et `float f=5.2; i=i%f;`

Il faudrait des règles différentes pour chaque identificateur, donc des milliards

$63^{32} > 10^{57}$

Irréaliste

Contrôle de types

$P \rightarrow D E$

$D \rightarrow T \text{ id} ; D$

$| \varepsilon$

$T \rightarrow B C$

$B \rightarrow \text{char}$

$| \text{int}$

$C \rightarrow [\text{num}] C$

$| \varepsilon$

$E \rightarrow \text{num}$

$| \text{literal}$

$| \text{id}$

$| E \text{ mod } E$

$| E [E]$

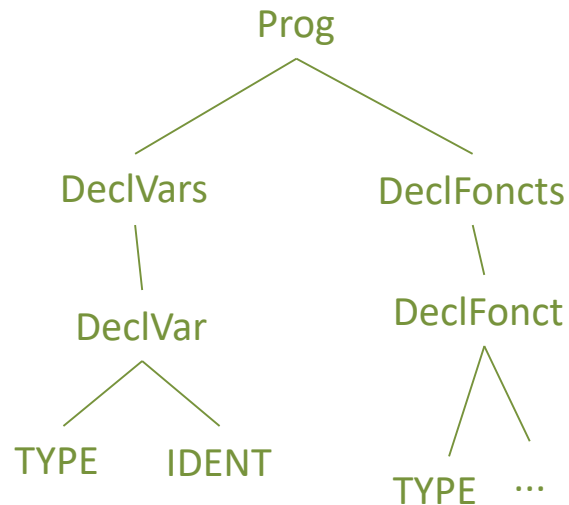
Avec cette grammaire, on peut déclarer une variable comme **char** et l'utiliser comme tableau de **int**

Le contrôle de types fait partie de l'analyse sémantique

Ajouter des vérifications et autres instructions

Placer ces ajouts dans le parcours de l'arbre abstrait

Contrôle de types



exemple d'arbre abstrait pour
`int count ; int (...)`

Contrôle de types

arbre abstrait pour

```
int count ;
char[64][32] msgs ;
msgs[count]
```

$P \rightarrow D E$

$D \rightarrow T id ; D$

$\mid \varepsilon$

$T \rightarrow B C$

$B \rightarrow \text{char}$

$\mid \text{int}$

$C \rightarrow [\text{num}] C$

$\mid \varepsilon$

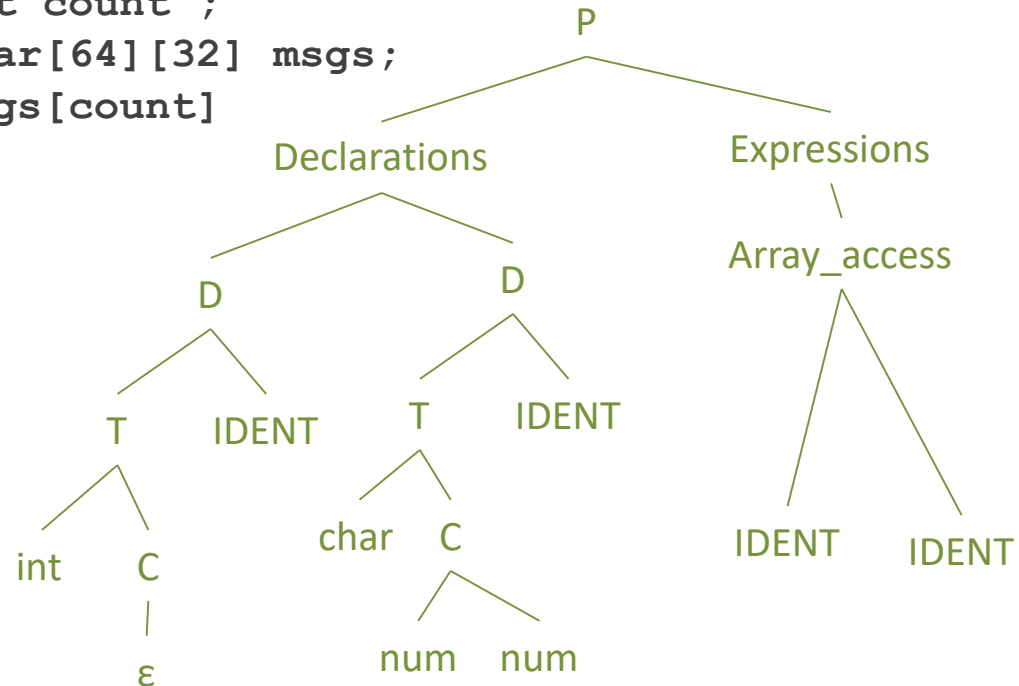
$E \rightarrow \text{num}$

$\mid \text{literal}$

$\mid \text{id}$

$\mid E \text{ mod } E$

$\mid E [E]$



literal : constante de type caractère

Déclarations

arbre abstrait pour

```
int count ;
char[64][32] msgs;
```

$P \rightarrow DE$

$D \rightarrow T \text{ id} ; D$

$\mid \varepsilon$

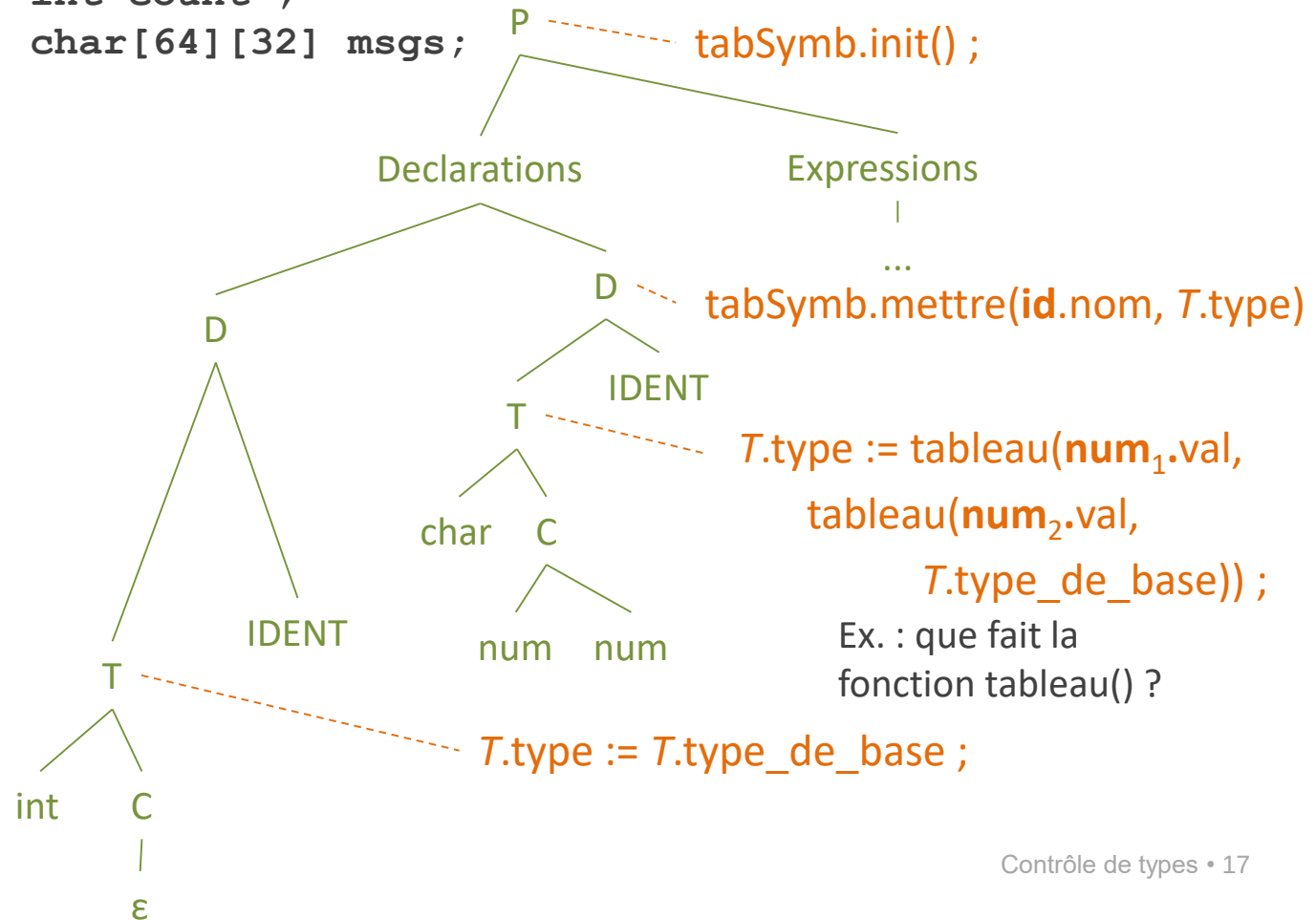
$T \rightarrow BC$

$B \rightarrow \text{char}$

$\mid \text{int}$

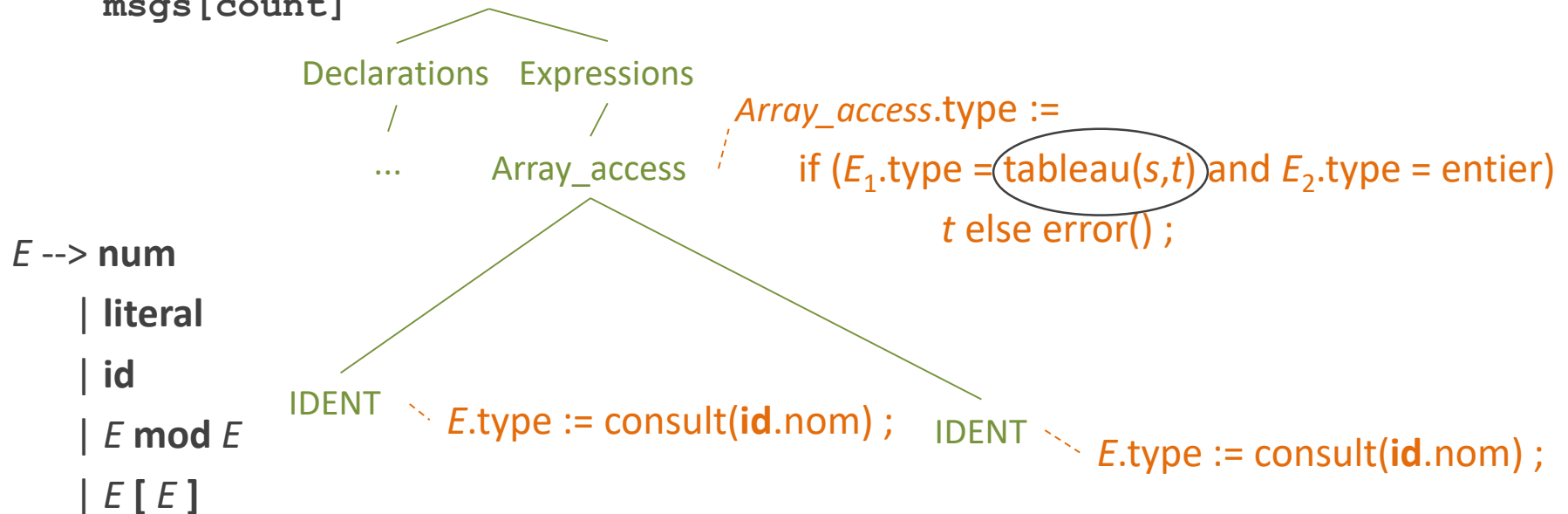
$C \rightarrow [\text{num}] C$

$\mid \varepsilon$



Expressions

arbre abstrait pour
msgs [count]



Comparaison entre $E.type$ et un arbre qui contient des variables ou des inconnues
La comparaison peut les initialiser

Instructions

$S \rightarrow \text{id} = E$	if ($\text{id.type} \neq E.\text{type}$) error() ;
$S \rightarrow \text{if } (E) S$	if ($E.\text{type} \neq \text{booléen}$) error() ;
$S \rightarrow \text{while } (E) S$	if ($E.\text{type} \neq \text{booléen}$) error() ;
$S \rightarrow S ; S$	

Instructions

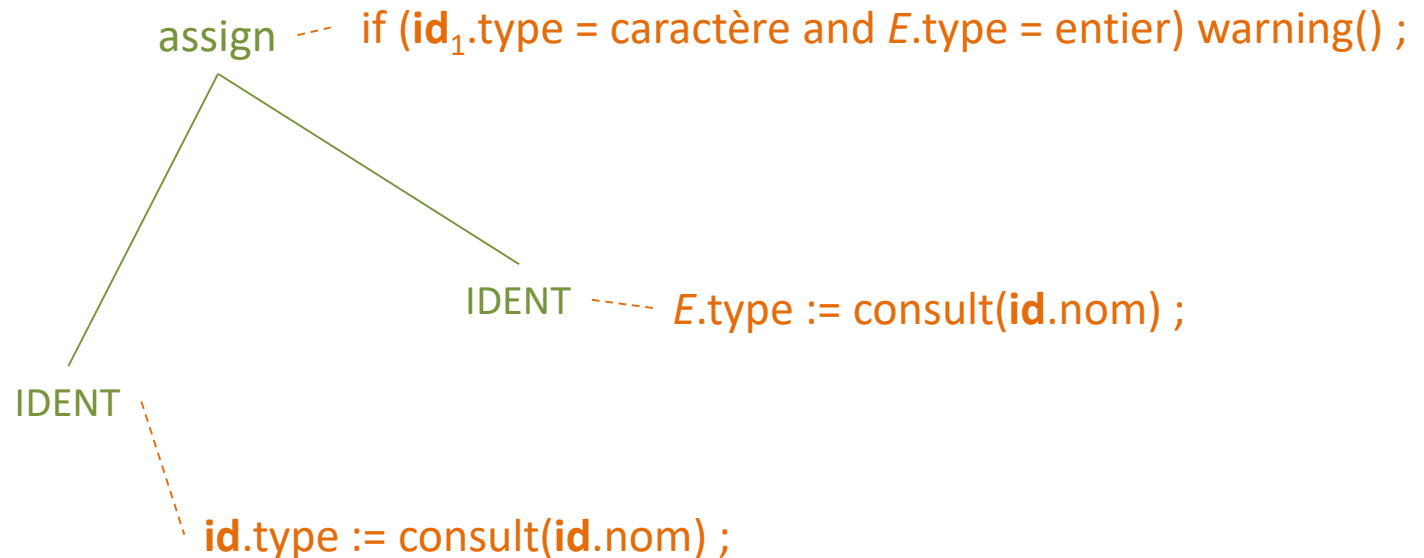
arbre abstrait pour
count=index

$S \rightarrow id = E$

$S \rightarrow \text{if } (E) S$

$S \rightarrow \text{while } (E) S$

$S \rightarrow S ; S$



Fonctions

$E \rightarrow E (E)$ $E.\text{type} := \text{if } (E_1.\text{type} = s \rightarrow t \text{ and } E_2.\text{type} = s)$
 $t \text{ else error() ;}$

Comparer deux types

Comment implémenter une comparaison qui dit si deux types sont équivalents ?

Exemples

Comparer paramètre formel et paramètre effectif

if ($E_1.type = s \rightarrow t$) and $E_2.type = s$)

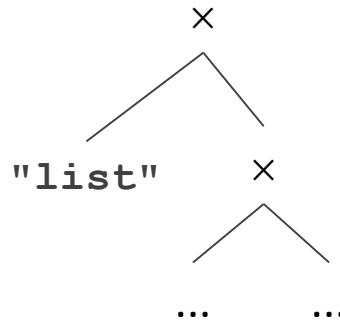
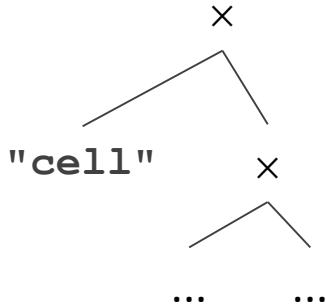
Vérifier le type d'un indice pour un tableau

if ($E_1.type = \text{tableau}(s, t)$) and $E_2.type = \text{entier}$)

Algorithme d'unification

On compare des arbres qui peuvent contenir des variables (s et t)

La comparaison peut les initialiser



Types nommés

```

typedef struct cell {
    int identifier ;
    struct cell * link ;
} cell ;
typedef struct list {
    int content ;
    struct list * next ;
} list ;
(...)
cell cell0 ;
list list0 ;

```

cell0 = list0 ; /* compile error */

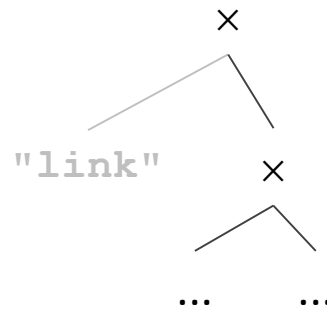
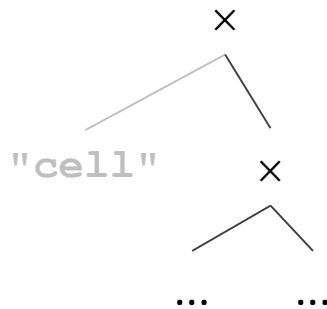
Quels sont les critères pour savoir si deux types nommés sont équivalents ?

Équivalence nominale

Le nom fait partie du type, et en plus on compare seulement le nom

C'est le cas en C

Mêmes types des champs, dans le même ordre, mais types différents



Types nommés

```
typedef struct cell {
    int identifiant ;
    struct cell * link ;
} cell ;
typedef cell list ;
(...)
cell cell0 ;
list list0 ;
cell0 = list0 ; /* OK */
```

Équivalence structurelle

Le nom ne fait pas partie du type, on compare seulement le reste de l'expression de type

Langages fonctionnels

Équivalence nominale large

En C, un typedef ne définit pas un type nouveau

Un nom introduit par typedef ne fait donc pas partie du type

Sommaire

Expressions de types

Un contrôleur de types

Types complexes en C

Conversions de types

Polymorphisme

Déclarations de types complexes en C

```
int * i, j;
```

{	int	type de base
	* i	déclarateur
	j	déclarateur

Déclarateur

Un déclarateur se comporte comme une expression : opérateurs, priorités entre opérateurs, parenthèses

La déclaration se fait à l'envers par rapport à l'expression de type

Commencer une déclaration de type complexe en C

```
int * i, j;
```

{	int	type de base
	* i	déclarateur
	j	déclarateur

pointeur
|
int

Pour lire ou construire une déclaration difficile en C

Commencer par lire ou construire le **déclarateur** et non le type simple

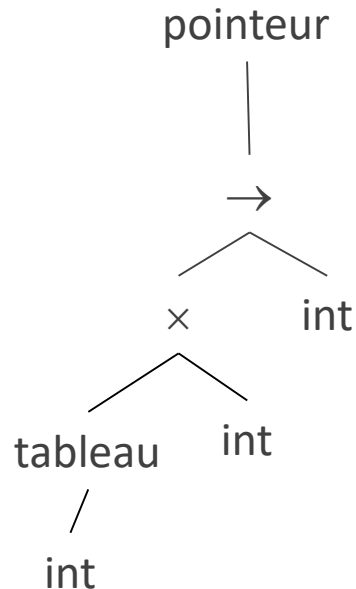
Un type complexe en C est toujours

- soit un tableau de quelque chose
- soit un pointeur sur quelque chose
- soit une fonction

C'est le constructeur de type à **la racine** de l'expression de type

Exemple

```
int * pointer...;
```



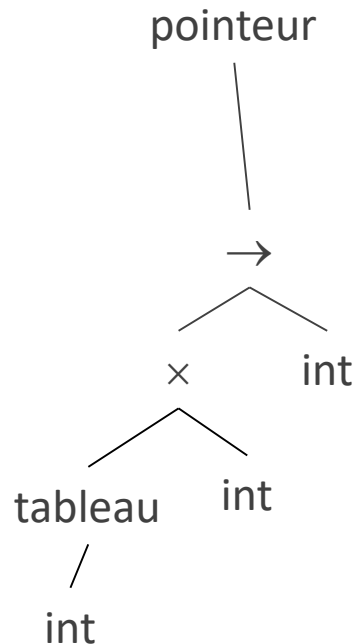
Pour indiquer le constructeur de type à la **racine** de l'expression de type

appliquer à l'identificateur qu'on veut déclarer

- les crochets si c'est un tableau
- le déréférencement (*) si c'est un pointeur
- les parenthèses d'appel si c'est une fonction

Continuer une déclaration de type complexe en C

```
int (* sorter)(int list[], int size);
```

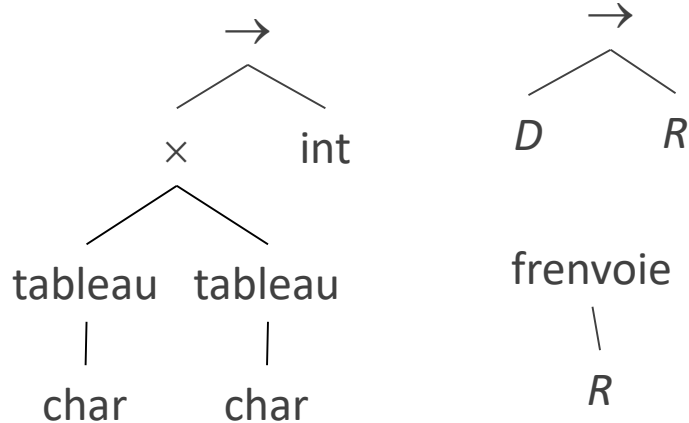


Pour indiquer

- le type des éléments d'un tableau
- sur quoi pointe un pointeur
- le type de retour d'une fonction,
penser à l'utilisation d'une variable ou d'une
fonction qui a ce type

Déclarations de types en C

```
int compare(char s1[], char s2[]);
```



Type fonction

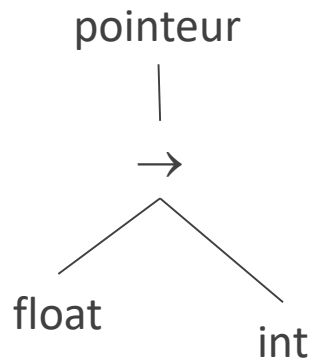
En C, un type fonction tient compte du type

- de la valeur de retour
- et des paramètres

Dans une déclaration de prototype, il est recommandé de préciser les paramètres (mais facultatif)

Déclarations de types en C

```
int (*myfunc)(float) ;
```



Type pointeur sur fonction

Obligatoire pour qu'une fonction soit

- élément de tableau
- paramètre d'une autre
- valeur de retour d'une autre

...

Déclarations de types en C

```
int tab[ ](float);  
/* non :  
tableau(→(float,int)) impossible */
```

```
int (* tab[ ])(float):  
/* oui :  
tableau(pointeur(→(float,int))) */
```

Type pointeur sur fonction

Pour pouvoir appliquer un constructeur de type à un **type fonction**, la syntaxe du C impose de construire d'abord un **type pointeur sur fonction**

Déclarations de types en C

```
int (* tab[])(float), count;
```

```
{
  int
  (* tab[])(float)
  count
}
```

type de base

déclarateur

déclarateur

Déclarateur

```
int (* tab[])(float)
```

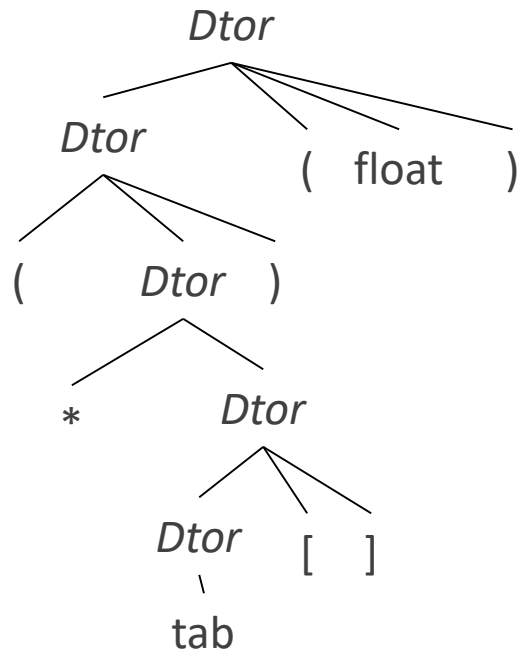
```
{
  int                                     type de base
  (* tab[])(float)                       déclarateur
}
```

La déclaration se fait à l'envers par rapport à
l'expression de type

L'arbre du déclarateur est à l'envers par rapport à
l'arbre de l'expression de type

Déclarations de types en C

int



déclarateur

marque de fonction

marque de pointeur

marque de tableau

tableau

pointeur

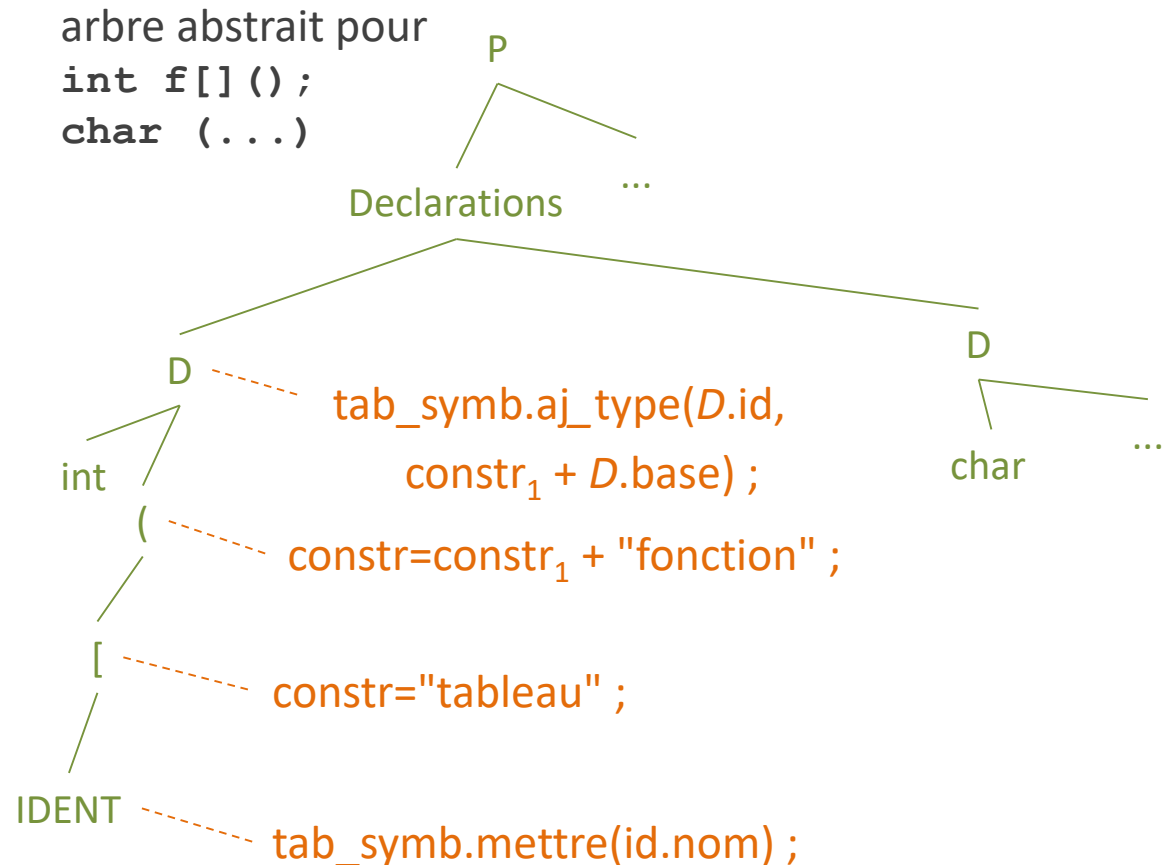
float

int

expression de type

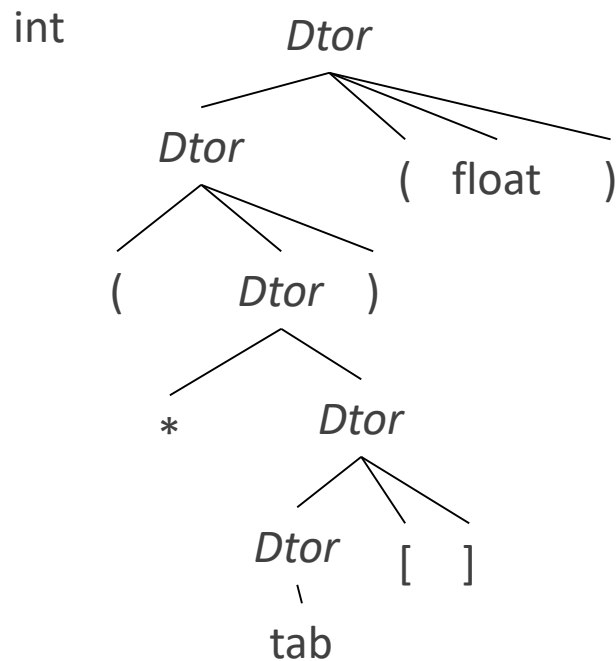
Traduire un déclarateur en expression de type

$P \rightarrow D ; E$
 $D \rightarrow D ; D$
 $D \rightarrow \text{base } D\text{tor}$
 $\text{base} \rightarrow \text{char}$
 $\text{base} \rightarrow \text{int}$
 $D\text{tor} \rightarrow D\text{tor} []$
 $D\text{tor} \rightarrow D\text{tor} ()$
 $D\text{tor} \rightarrow \text{id}$
 $D\text{tor} \rightarrow * D\text{tor}$
 $D\text{tor} \rightarrow (D\text{tor})$



"+" représente la concaténation de chaînes de caractères

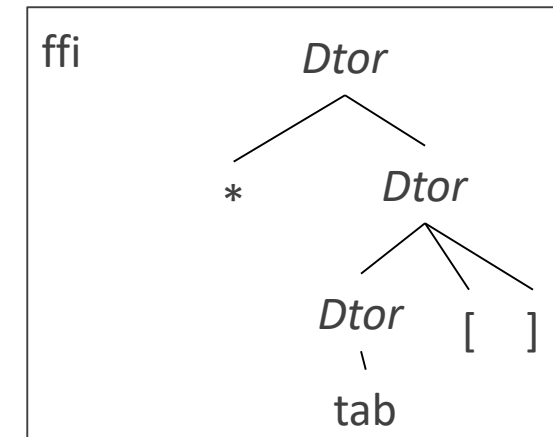
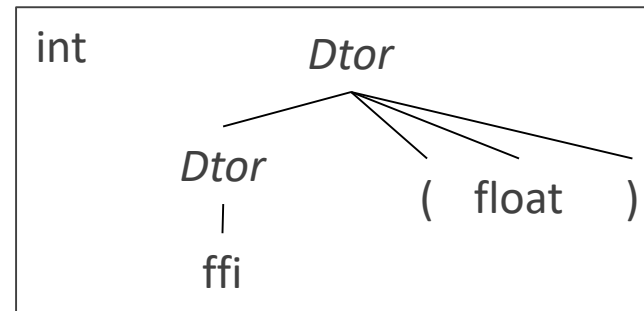
Avec typedef



```
typedef int (*tpffi[5])(float);
```

```
tpffi tab;
```

Introduit un nouveau nom (**tpffi**) équivalent au type déclaré



On peut couper en deux une déclaration de type compliquée en mettant le haut du déclarateur dans un typedef

```
typedef int ffi(float);
ffi *tab[5];
```

Exercice : couper en deux une déclaration compliquée avec un typedef

Grammaire

Pour déclarer plusieurs variables avec le même
type de base

D --> *base liste*

liste --> *Dtor*

liste --> *liste , Dtor*

Sommaire

Expressions de types

Un contrôleur de types

Déclarations de types en C

Conversions de types

Polymorphisme

Conversions de types

Traduction en code intermédiaire :

tmp := intToReal i

tmp := real+ x tmp

où **real+** désigne l'addition des réels

L'expression **x + i** où **x** est un réel et **i** un entier nécessite une conversion

L'addition des réels et l'addition des entiers sont des opérations distinctes car la représentation des données est distincte

Conversions implicites (*coercion*)

Sans risque de perte d'informations

Faites par le compilateur

Conversions explicites (*casting*)

Exemples en C : **(float) 10**

(int) PI

Conversions implicites

$$E \rightarrow \text{num}$$

```
E.type := entier ;
```

$$E \rightarrow \text{num.num}$$

$E.type := \text{réel} ;$

$$E \dashrightarrow \text{id}$$

```
E.type := lookup(id.entry);
```

$$E \rightarrow E \text{ op } E$$
$$E.type := \text{if } (E_1.type = \text{entier} \text{ and } E_2.type = \text{entier})$$

entier ;

else if ($E_1.type=entier$ and $E_2.type=réel$)

réel ;

etc.

Sommaire

Expressions de types

Un contrôleur de types

Déclarations de types en C

Conversions de types

Polymorphisme

Espaces de noms

Paramètres et variables locales d'une fonction

Tous ont des noms différents

Une seule table des symboles : facilite la détection
des redéclarations

Fonctions et variables globales

Tous ont des noms différents

Espace de noms

Ensemble de noms d'objets dans lequel tous les
objets ont des noms différents

En C, un espace de noms
global et un pour chaque
bloc

Polymorphisme

Surcharge (*overloading*)

Un symbole sert à un nombre fini de choses
Si le symbole est un nom, il est partagé par un
nombre fini d'objets associés dans un même
espace de noms

Types génériques (*generic types*)

Fonctions sur les types

Surcharge

Cas où un même symbole représente plusieurs opérateurs

L'opérateur à appliquer est choisi en fonction du contexte

Exemple : + désigne

- l'addition des entiers
- l'addition des matrices
- l'addition d'un entier à un caractère

De même pour les fonctions

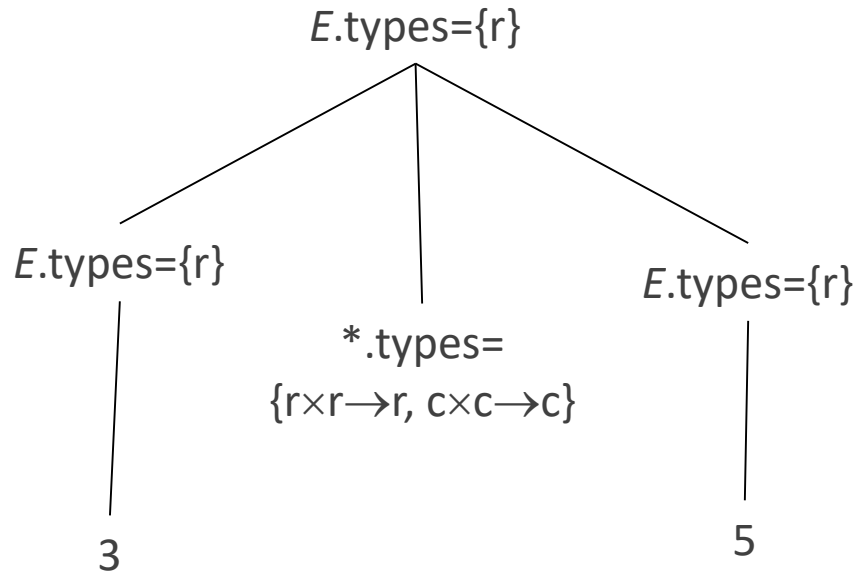
Dans le projet, toutes les surcharges sont résolues syntaxiquement

Grammaire attribuée pour calculer les types possibles d'une expression

$E \rightarrow \text{id}$ $E.\text{types} := \text{lookup}(\text{id}.\text{entry})$

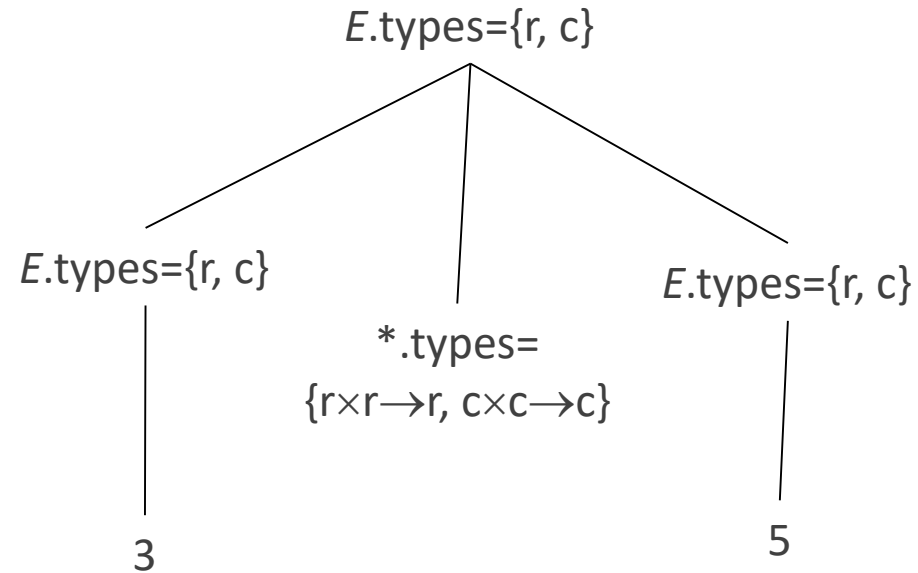
$E \rightarrow E (E)$ $E.\text{types} := \{ t \mid \text{il existe un } s \text{ dans } E_2.\text{types} \text{ tel que } s \rightarrow t \text{ est dans } E_1.\text{types} \}$

Surcharge des opérateurs



Exemple : l'opérateur $*$ peut prendre des opérandes réels ou complexes et le résultat peut être réel ou complexe

Surcharge des opérateurs



Surcharge des méthodes en Java

```
class A {  
    int a, b;  
    A(int a, int b){this.a = a; this.b = b; }  
}  
class B{  
    private A couple;  
    B(int a, int b){couple = new A(a,b); }  
    B(A couple){this.couple = new A(couple.a, couple.b); }  
    A getA( ){ return couple;}  
    void ajouter(int inc){getA().a += inc; getA().b += inc; }  
    void ajouter(B couple){  
        this.getA().a += couple.getA().a;  
        this.getA().b += couple.getA().b; }  
}
```


Types génériques

Fonction générique : dont les paramètres ou la valeur de retour sont de type non précisé

Exemples

L'opérateur **&** du C : si E est de type t , alors **&** E est de type `pointer(t)`

Les fonctions génériques du langage ADA

Certaines fonctions en langage CAML

Exemple non générique

```
typedef struct cell { int info ; struct cell * link ; } cell ;  
int length(cell * list) {  
    int len = 0 ;  
    while (list) { len ++ ; list = list -> link ; }  
    return len ; }
```

Calcul de la longueur d'une liste

On est obligé de connaître le type des éléments de la liste pour déclarer le type cell et la fonction

Exemple générique

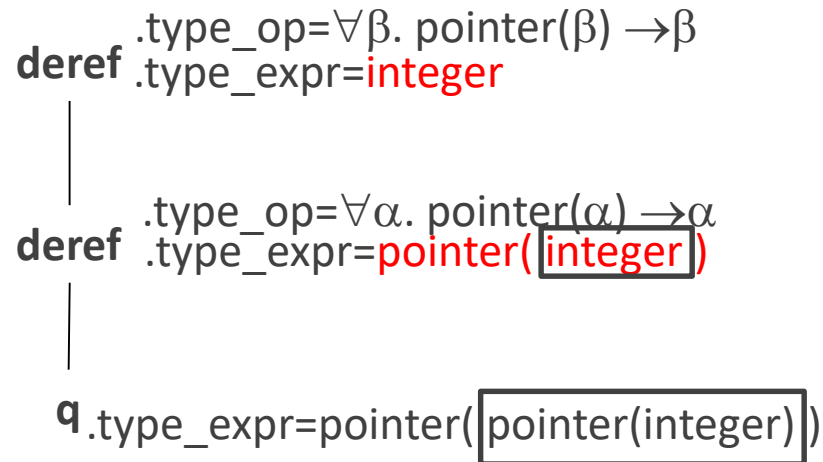
En ML

```
fun length(list) =  
    if null(list) then 0  
    else length(tl(list)) + 1 ;
```

`null()` et `tl()` sont prédéfinies

Un langage avec généricité

$$\begin{aligned}
 P &\rightarrow D ; E \\
 D &\rightarrow D ; D \mid \text{id} : Q \\
 Q &\rightarrow \forall \text{typeVar} . Q \mid T \\
 T &\rightarrow \text{basicType} \\
 &\quad \mid \text{unaryConstructor} (T) \\
 &\quad \mid \text{typeVar} \\
 &\quad \mid T \rightarrow T \\
 &\quad \mid T \times T \\
 &\quad \mid (T) \\
 E &\rightarrow E (E) \mid E , E \mid \text{id}
 \end{aligned}$$



Exemple

Arbre syntaxique de l'expression `deref(deref(q))`

Déclaration d'un pointeur

`q : pointer(pointer(integer)) ;`

Déclaration de la fonction de déréférencement

`deref :  $\forall \alpha . \text{pointer}(\alpha) \rightarrow \alpha$  ;`

deref $.type_op = \forall \beta. \text{pointer}(\beta) \rightarrow \beta$
 $.type_expr = \text{integer}$

deref $.type_op = \forall \alpha. \text{pointer}(\alpha) \rightarrow \alpha$
 $.type_expr = \text{pointer}(\text{integer})$

q.type = $\text{pointer}(\text{pointer}(\text{integer}))$

Exemple

Des occurrences distinctes d'une fonction
générique n'ont pas nécessairement le même
type

$\text{pointer}(\text{pointer}(\text{integer})) \rightarrow \text{pointer}(\text{integer})$

$\text{pointer}(\text{integer}) \rightarrow \text{integer}$

Le calcul des types des variables fait appel à un
algorithme d'**unification**

Typage dynamique

- En cas d'héritage, un objet peut avoir plusieurs types, certains plus précis que d'autres
- Un type connu à l'exécution peut être plus précis que le type calculable à la compilation, parce qu'à l'exécution on sait par quelle partie du code on est passé
- À l'exécution, chaque objet est stocké sous la forme d'une valeur et d'un type

Merci

CONTACT

ERIC LAPORTE

+33 1 60 95 75 52

ERIC.LAPORTE@UNIV-EIFFEL.FR